

From: <https://chatgpt.com/c/6ac7a6d4-d8fc-83e8-9285-0b9e396f2d26>

you asked: this is my

message time: 2026-10-08 19:51:15

this is my se lab 4 give how to do this and attached **individual** project github link Please clone the team repository directly rather than forking it, as cloning is the recommended approach for our project.

https://github.com/SETAPESU26/61_math_flashcards

chatgpt response: Yes . I

Yes . I checked your **Lab 4 handout** and the GitHub repository you provided. This is an **individual Vibe Coding assignment**, and the main requirement is to clone the assigned team repo, fix/add features using AI prompts, make separate commits, and push your completed work to your **own GitHub repository**.

What you need to do

Your assigned repository is:

The workflow should be:

text

Team Repository

↓

git clone

↓

Run existing project

↓

Record 10-sec BEFORE video

↓

Read README

↓

Use Vibe Coding / ChatGPT

↓

Fix broken code

↓

Add README features one-by-one

↓

```
Separate commit for each task
↓
Push to YOUR GitHub repo
↓
Record 10-sec AFTER video
↓
Create Lab-4 folder
↓
Upload videos + code + Chat history PDF
```

This follows the handout instructions, including the requirement for separate commits and **no PR** to SETAPESU26.

Step 1 — Clone the repository

Open **VS Code**.

Open Terminal:

Terminal → New Terminal

Run:

```
bash
git clone https://github.com/SETAPESU26/61_math_flashcards.git
```

Then:

```
bash
cd 61_math_flashcards
```

Check the files:

```
bash
dir
```

You should see the project files.

Then open it in VS Code:

```
bash
code .
```

Step 2 — Read the README

This is **very important**.

Run:

```
bash
type README.md
```

Or simply open:

```
text
README.md
```

The lab specifically says to go through the README to understand the deliverables and then add the features listed there.

Don't start modifying the code before checking the README.

Step 3 — Install requirements

Look for:

```
text
requirements.txt
```

If it exists:

```
bash
pip install -r requirements.txt
```

If `pip` doesn't work:

```
bash
python -m pip install -r requirements.txt
```

Step 4 — Run the original project

Because this is a Python game/puzzle assignment, first identify the main Python file.

Run:

```
bash
dir *.py
```

For example, if you see:

```
text
app.py
```

run:

```
bash
python app.py
```

If you see:

```
text
main.py
```

run:

```
bash
python main.py
```

Important

At this stage **do not change anything**.

You need to show the original/broken version first.

Step 5 — Record the BEFORE video

The handout specifically requires:

Record 10 seconds of video before making any changes.

So record approximately **10 seconds** showing:

- Project running
- Existing UI/game
- Broken functionality, if visible
- Terminal/project

On Windows you can use:

```
text
Win + G
```

and start recording.

Save it as:

```
text
```

```
before.mp4
```

Step 6 — Now use Vibe Coding

This is the main part of the lab.

You shouldn't manually rewrite everything.

Give the project/code to ChatGPT and use prompts to:

1. Understand the existing code
2. Find bugs
3. Fix bugs
4. Add each required feature
5. Test the result

The handout's objective is specifically to learn how to prompt Vibe Coding tools and get the task done in around **3–4 attempts**.

Step 7 — First prompt: analyze the project

After you have the project open, give ChatGPT the relevant files or paste the code.

Use a prompt like:

```
text
```

```
I am doing a Software Engineering Lab 4 Vibe Coding assignment.
```

```
This is an individual project based on the repository:
```

```
61_math_flashcards
```

```
First, analyze the existing project without making unnecessary changes.
```

```
Read the code and README carefully.
```

```
Identify:
```

1. What the application currently does
2. The existing features
3. Broken or incomplete functionality
4. Errors or bugs
5. All features requested in the README

```
6. Which files need to be modified
```

```
Do not rewrite the entire project.
```

```
Do not remove existing functionality.
```

```
Give me a clear task-by-task implementation plan first.
```

Step 8 — Fix the broken code

After ChatGPT identifies the problems, use:

```
text
```

```
Now fix the existing broken functionality.
```

```
Requirements:
```

- Modify only the necessary files
- Preserve the existing project structure
- Do not remove working features
- Do not introduce unnecessary dependencies
- Keep the implementation simple and reliable
- Make sure the application runs without errors

```
After making the changes, explain exactly what was fixed and how I can test it.
```

Then test it in VS Code.

Step 9 — Add README features one by one

This is important because the lab explicitly says to write prompts to add **all features listed in the README one by one**.

Don't ask:

```
"Add everything."
```

Instead do:

Feature 1

```
text
```

```
Implement Feature 1 from the README.
```

```
Requirements:
```

- Keep all existing functionality
- Implement only this feature
- Follow the existing project architecture
- Do not modify unrelated code
- Handle invalid input properly
- Make the UI clear and user-friendly

After implementation, tell me:

1. Which files changed
2. What was implemented
3. How to test it

Test it.

Then commit it.

Step 10 — Make separate commits

The handout specifically says:

Have Separate Commits for each task.

For example:

Commit 1 — Bug fix

```
bash
git add .
git commit -m "Fix existing flashcard functionality"
```

Commit 2 — Feature 1

```
bash
git add .
git commit -m "Add flashcard generation feature"
```

Commit 3 — Feature 2

```
bash
git add .
git commit -m "Add answer validation"
```

Commit 4 — Feature 3

```
bash
git add .
git commit -m "Add score tracking"
```

Your actual commit messages should match the features in your README.

Lab 4 – Task 1: Fix the String Concatenation Bug

1. Objective

Fix the arithmetic calculation bug in `game/game\_engine.py`. The original game incorrectly joined the operands as text instead of performing the mathematical operation specified by the active operator.

2. Original code before modification

Open `game/game\_engine.py` and locate this function:

```
```py
3. Explanation of the bug
```

Suppose the flashcard displays:

```
```he original function evaluates:
```

```
```pcause the values are inserted into a formatted string, the result is `"86"`.
Converting it to an integer produces `86`, not `14`.
```

Therefore, entering `14` is incorrectly rejected.

The function also ignores the active operator, so it cannot correctly handle subtraction or multiplication.

#### ## 4. Corrected code

Replace the original `compute\_expected\_answer()` function with:

```
```pythovalues the operands numerically and selects the calculation according to the
current operator.
```

5. What changed?

Before	After
Operands joined as text	Operands used in arithmetic
Operator ignored	Active operator determines the result
`8 + 6` gives expected answer `86`	`8 + 6` gives `14`
Correct answers rejected	Correct answers accepted

The existing subtraction safety logic remains unchanged.

6. Testing Task 1

Run the project from the terminal:

```
```bash
python main.pcases:

| Operation | Question | Input | Expected result |
|---|---|---:|---|
```

```
| Addition | `8 + 6` | `14` | Correct |
| Subtraction | `9 - 4` | `5` | Correct |
| Multiplication | `6 * 3` | `18` | Correct |
| Incorrect answer | `8 + 6` | `15` | Wrong; expected 14 |
```

The application should display the correct-answer feedback for valid answers and the expected value for incorrect answers.

```
7. Git commit for Task 1
```

After testing, the change was committed separately using:

```
```bash  
git add game/game_engine.py  
git com--oneline
```

Your recordedf0d7fb2 Fix arithmetic calculation burt

<WritingBlock id="62714" variant="document">##r chat history

Bro, this is a reconstructed Task 1 record, not a complete export of the original conversation. Since your README requests the complete Chat/LLM history, keep the actual conversation containing your debugging discussion and changes. You can use ChatGPT's conversation sharing/export options to preserve that original history rather than presenting this summary as the full transcript.

you asked: give task

message time: 2026-10-09 09:06:07

give task 2

chatgpt response: Bro, here

Bro, here is **Task 2 — Implement a Per-Question Timer Bar**, in the same format as Task 1, for your Lab 4 chat-history documentation.

Lab 4 — Task 2: Implement a Per-Question Timer Bar

1. Objective

Implement a countdown timer for every math flashcard in the Pygame application. Each question should have a fixed time limit of 10 seconds. When the timer expires, the game should automatically register a missed attempt and move to the next question.

2. Original code before modification

In `game/game_engine.py`, the original `update()` method was:

```
here
```

```
## 3. Changes made
```

Change 1: Add timer variables

Inside `__init__()`, after the score and attempt variables, add:

```pyse variables store the time limit, remaining time, last update timestamp, and scheduled time for displaying correct-answer feedback.

### Change 2: Reset the timer for each new card

Inside `generate_new_card()`, add the timer reset logic:

```pytever a new question is generated, its timer resets to 10 seconds.

Change 3: Implement countdown and timeout handling

Replace the original `update()` method with:

```pythcalculates elapsed time using Pygame's millisecond clock, subtracts it from the remaining time, and registers a missed attempt when the timer reaches zero.

### Change 4: Draw the timer bar

Inside `render()`, add the following after:

```pythofore drawing the Submit button:

```pythonk rectangle represents the timer's full width, while the green rectangle represents the remaining time.

### Change 5: Preserve correct-answer feedback

To prevent the correct-answer message from disappearing immediately, the `submit_answer()` method schedules the next card after a short delay:

```python

ate() method waits until that timestamp before generating the next question.

The main game loop in `main.py` already calls `engine.update()` and `engine.render(screen)`, so no changes to `main.py` were necessary.

4. Testing Task 2

Run the game:

```
```bash
```

```
python3 game.py
```

```
|---|---|
```

```
| Timer starts | Full green bar is visible |
```

```
| Time passes | Green bar gradually shrinks |
```

```
| Correct answer | Feedback appears, then the next question loads with a reset timer
```

```
|
```

```
| Wrong answer | Wrong-answer feedback appears; the current question remains |
```

```
| Empty answer | Warning appears without increasing attempts |
```

```
| Timeout | Attempt count increases and a new question appears |
```

#### ## 5. Git commit

After testing the timer, the changes were committed separately:

```
```bash
```

```
git add game/game.py
```

```
git log --oneline
```

```
```text
```

```
a3896b1 Add per-question timer
```

<WritingBlock id="58327" v1:1> Task 2 was committed separately from Task 1, preserving the incremental development history required for the lab.

```

```

```
you asked: give task
```

```
message time: 2026-10-09 09:06:26
```

```
give task 3
```

```

```

```
chatgpt response: Bro, here
```

```
Bro, here is Task 3 – Implement Consecutive Correct Streak Multipliers, in the same format as Tasks 1 and 2 for your Software Engineering Lab 4 documentation.
```

```
Lab 4 – Task 3: Implement Consecutive Correct Streak Multipliers
```

```
1. Objective
```

```
Implement a streak-based scoring system in the Math Flashcards Pygame application. Players earn increasing score rewards for consecutive correct answers. The streak and multiplier reset when the player submits a wrong answer or the timer expires.
```

```
An empty submission must not count as a failed attempt or reset the streak.
```

```
2. Original code before modification
```

```
In `game/game_engine.py`, the original correct-answer logic awarded one point for every correct answer:
```

```
```he game had no streak counter or multiplier, so consecutive correct answers received the same reward.
```

```
## 3. Changes made
```

```
### Change 1: Add streak variables
```

```
Inside `__init__()` , after the score and attempt variables, add:
```

```
```pystreak` tracks the number of consecutive correct answers.
```

```
- `streak_multiplier` determines the points awarded for the next correct answer.
```

```
Change 2: Update correct-answer scoring
```

Replace the original correct-answer scoring block with:

```
```python
consecutive correct answer increases the multiplier and adds the corresponding
points to the score.
```

For example:

```
| Consecutive correct answer | Multiplier | Points earned | Total score |
|---|---:|---:|---:|
| First | x1 | +1 | 1 |
| Second | x2 | +2 | 3 |
| Third | x3 | +3 | 6 |
| Fourth | x4 | +4 | 10 |
```

Change 3: Reset the streak on a wrong answer

Update the wrong-answer block:

```
```python
else:
 self.strethe streak returns to zero and the multiplier returns to x1.
```

### Change 4: Reset the streak on timeout

Inside `update()`, add the reset logic to the timeout handling:

```
```python
if self.time_remainirecords a missed attempt, resets the streak, and generates
another question.
```

Change 5: Preserve empty-input behavior

The empty-input validation remains unchanged:

```
```python
if not val_str or valefore the scoring and streak-reset logic, an empty submission
does not increase the attempt count or reset the streak.
```

```
4. Testing Task 3
```

Run the application:

```
```bash  
python main.py
```

Test | Test | Expected behavior | |---|---| | First correct answer | Streak 1, multiplier $\times 1$, score +1 | | Second consecutive correct answer | Streak 2, multiplier $\times 2$, score +2 | | Third consecutive correct answer | Streak 3, multiplier $\times 3$, score +3 | | Wrong answer | Streak resets to 0; multiplier returns to $\times 1$ | | Timeout | Streak resets to 0; multiplier returns to $\times 1$ | | Empty submission | Attempt count and streak remain unchanged |

5. Git commit

After implementing and testing Task 3, the changes were committed separately:

```
bash  
git add game/game_engine.py  
git commit
```

Your recorded Tab261 Add consecutive correct streak mtingBlock id="73519" variant="document">### Tted separately from Tasks 1 and 2, preserving the incremental development history.

you asked: gve task 4

message time: 2026-10-09 09:06:46

gve task 4

chatgpt response: Bro, here

Bro, here is **Task 4 — Implement Division Operator with Clean Integer Quotients**, in the same format as Tasks 1, 2, and 3 for your Software Engineering Lab 4 documentation.

Lab 4 — Task 4: Implement Division Operator with Clean Integer Quotients

1. Objective

Extend the Math Flashcards Pygame application by adding the division ($\div$) operator.

The division questions must satisfy these requirements:

- Generate division questions randomly alongside addition, subtraction, and multiplication.
- Every division question must produce a whole-number answer.
- Division must never produce a remainder.
- Division by zero must never occur.
- Preserve the existing timer, streak multiplier, scoring, and feedback functionality.

2. Original code before modification

In `game/game_engine.py`, the original `generate_new_card()` method selected only three arithmetic operators:

```
py

```pyrefore, division questions were never generated or evaluated.

3. Changes made

Change 1: Add division to random operator selection

Replace:

```pyth

```pythoe game randomly selects from four arithmetic operations.

Change 2: Generate valid division questions

Replace the existing `generate_new_card()` method with:

```pythonnation:**

Instead of randomly generating two numbers and hoping they divide evenly, the program
first selects a divisor and an integer answer.

It then calculates:

$$
\text{Dividend} = \text{Divisor} \times \text{Answer}
$$
```

For example:

```
```text
```

Divintees that every division question has a whole-number answer without a remainder.

```
Change 3: Update the answer calculation
```

Replace `compute_expected_answer()` with:

```
```python
```

deferator performs integer floor division in Python. Since the generated dividend is always an exact multiple of the divisor, the quotient is mathematically exact.

```
## 4. Testing Task 4
```

Run the application:

```
```bash
```

python owing cases:

Test   Expected behavior
--- ---
Addition   Correct arithmetic result
Subtraction   Correct result without negative generated answers
Multiplication   Correct arithmetic result
Division   Whole-number quotient
Division by zero   Never generated
Non-integer quotient   Never generated
Timer   Continues working
Streak multiplier   Continues working
Wrong answer   Displays expected answer
Timeout   Records missed attempt and resets streak

```
Example valid division questions
```

```
```text
```

24 / 6 = 4

30 / 5 stions prevented

```
```text
10 / 3
7 / 2
5 / 0d by the new division-generation logic.
```

## ## 5. Git commit

After implementing and testing Task 4, the changes were committed separately:

```
```bash
git add game/game_enh
git log --oneline
``` was:
```

```
```text
0498450 Add integer di
```text
0498450 Add integer div lab report
```

<WritingBlock id="73520" variant 4 completion status

Task	Git commit	Status
Task 1 – Arithmetic bug fix	<code>`f0d7fb2`</code>	 Completed
Task 2 – Countdown timer	<code>`a3896b1`</code>	 Completed
Task 3 – Streak multiplier	<code>`d0cb261`</code>	 Completed
Task 4 – Integer division	<code>`0498450`</code>	 Completed

**\*\*Important for submission:\*\*** Tasks 1-4 above are documentation summaries, not a verbatim chat export. Since your lab specifically requests the complete LLM conversation history, submit the actual ChatGPT shared conversation link as well as your before-and-after gameplay videos.